1. Define Algorithm & its Characteristics

✅ **Definition**

An **algorithm** is a **step-by-step procedure** to solve a problem in a finite amount of time.

✅ **Characteristics**

1. **Input** → Takes zero or more inputs
2. **Output** → Produces at least one output
3. **Finiteness** → Must terminate after finite steps
4. **Definiteness** → Steps must be clear and unambiguous
5. **Effectiveness** → Steps must be simple and executable

---

2. Greedy & Divide and Conquer

🔷 **Greedy Algorithm**

- Chooses **best immediate option**
- Example: Knapsack, Activity Selection

🔷 **Divide & Conquer**

- Divide → Solve → Combine
- Example: Merge Sort, Quick Sort

✅ **Difference**

- Greedy: No reconsideration
- Divide & Conquer: Recursive breakdown

---

3. Difference Between Program & Algorithm

Algorithm	Program
Step-by-step logic	Implementation in code
Language independent	Language dependent
Easier to understand	Needs syntax
Focus on solution	Focus on execution

---

4. Knapsack Problem + Greedy Algorithm

🎯 **Problem**

Given weights & values, maximize value without exceeding capacity.

🔷 **Greedy Approach (Fractional Knapsack)**

1. Compute **value/weight ratio**
2. Sort items in decreasing order
3. Pick items fully or fractionally

✅ **Example**

Capacity = 50
Items:

- (value=60, weight=10) → ratio=6
- (100,20) → 5
- (120,30) → 4

Pick → 60 + 100 + (20/30 × 120) = **240**

---

5. Asymptotic Notation

It is used to **measure algorithm efficiency** for large inputs.

👉 Shows how time/space grows with input size.

---

6. Big O, Ω, Θ Notation

🔷 **Big O (O)**

- Worst case
- Upper bound
  👉 Example: $O(n^2)$

🔷 **Big Omega (Ω)**

- Best case
- Lower bound

🔷 **Big Theta (Θ)**

- Exact bound
  👉 Both upper & lower

---

7. Recurrence Relation & Solving Methods

🔷 **Recurrence Relation**

Defines a problem in terms of smaller inputs
👉 Example:
T(n) = 2T(n/2) + n

🔷 **Methods to Solve**

1. Substitution Method

- Guess solution & prove by induction

2. Recursion Tree

- Expand and sum all levels

3. Master Theorem

- Direct formula-based solution

---

8. Master's Theorem

Used to solve recurrence of form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

✅ **Cases**

1. **If** f(n) < n^logb(a) → T(n) = Θ(n^logb(a))
2. **If** f(n) = n^logb(a) → T(n) = Θ(n^logb(a) log n)
3. **If** f(n) > n^logb(a) → T(n) = Θ(f(n))

---

9. Solve using Master's Theorem

Example:

T(n) = 2T(n/2) + n

- a = 2, b = 2
- logb(a) = $\log_2 2$ = 1
- f(n) = n

👉 Case 2 applies

✅ **Answer**

T(n) = Θ(n log n)

---

10. Time Complexity Table (IMPORTANT)

🔷 **Sorting Algorithms**

Algorithm	Best	Average	Worst
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$